

Introduction à la programmation en Java

Algorithmique, structures de contrôle et méthodes

Cours d'initiation à la programmation

Pr. Khadija SLIMANI

khadija.slimani@esiea.fr

PASS Java — Année universitaire 2026–2027

Plan du cours

Séance 1 — Les fondamentaux de la programmation Java

1. Introduction à l'algorithmique
2. Structure de base d'un programme Java
3. Variables
 - Règles de nommage
 - Déclaration et initialisation
 - Types de données
 - Affectation
 - Conversion
 - Visibilité d'une variable dans un bloc
4. Expressions et opérateurs
5. Structures conditionnelles
6. Structures répétitives : boucles

Séance 2 — Méthodes en Java

7. Méthodes en Java
 - Qu'est-ce qu'une méthode ?
 - Déclaration d'une méthode
 - Appel d'une méthode
 - Paramètres
 - Valeur de retour
 - void ou valeur de retour ?
 - Portée des variables
 - Exemple complet
8. Exercices d'application

Séance 1

Les fondamentaux de la programmation Java

De l'algorithme aux structures de contrôle

Introduction à l'algorithmique

- **Qu'est-ce qu'un algorithme ?**

Un algorithme est une suite finie et ordonnée d'instructions permettant de résoudre un problème.

Exemple simple :

- **Problème** : calculer la moyenne de trois nombres.

Algorithme :

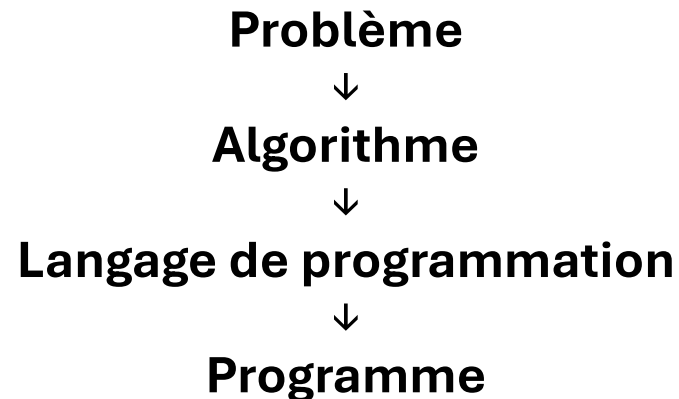
- Lire les trois nombres.
- Calculer leur somme.
- Diviser la somme par 3.
- Afficher la moyenne.

Introduction à l'algorithmique

- **De l'algorithme à la programmation**

Un algorithme décrit comment résoudre un problème.

La programmation consiste à traduire cet algorithme dans un langage de programmation.



Langage	Exemple
Python	<code>print("Bonjour")</code>
C	<code>printf("Bonjour");</code>
Java	<code>System.out.println("Bonjour");</code>

Introduction à l'algorithmique

- Pourquoi utiliser Java ?
 - Java est un **langage de programmation**.
 - Il permet de transformer un algorithme en **programme exécutable**.
 - Il est largement utilisé dans le développement logiciel.
 - Sa syntaxe permet d'apprendre les concepts fondamentaux de la programmation.
 - Il est **orienté objet**, notion qui sera abordée progressivement.

Structure de base d'un programme Java

- Les **commentaires** sont des textes ajoutés dans le programme pour expliquer le code. Ils sont ignorés par le compilateur et ne sont pas exécutés.

```
1 | // la suite est un commentaire
```

```
1 | /* ceci est un commentaire  
2 | sur plusieurs  
3 | lignes */
```

- Une **instruction** est une phrase qui respecte la grammaire de Java, elle se termine toujours par un point virgule ;

```
1 | short population;  
2 | population = 30000;
```

```
1 | short population = 1.000.000;
```

```
1 | long nbParticules = 10.000.000.000;
```

Structure de base d'un programme Java

- **Premier programme Java**

```
1 public class Bonjour {  
2     public static void main(String[] args) {  
3         System.out.println("Bonjour !");  
4     }  
5 }
```

Variables

○ Règles de nommage

- Le nom d'une variable doit commencer par une lettre.
- Il peut contenir des lettres, des chiffres, _ et \$.
- Même si Java autorise certains caractères accentués, il est recommandé d'utiliser des caractères non accentués pour éviter les problèmes d'encodage.

Variables

○ Déclaration et initialisation

- *déclaration simple :*

`<type> <nom>;`

- *déclaration avec affectation :*

`<type> <nom> = <valeur dans le type> | <variable> | <expression>;`

- *déclaration multiple :*

`<type> <nom1>, <nom2>, ..., <nomk>;`

- *déclaration multiple avec affectation partielle :*

`<type> <nom1>, <nom2>= <valeur dans le type>, ..., <nomk>;`

Variables

- Types de données

- Deux grands groupes de types de données :

- types primitifs
 - objets (instances de classe)

- Java manipule différemment les valeurs des types primitifs et les objets; les variables contiennent des valeurs de types primitifs ou des références aux objets

Variables

- Types de données : Types primitifs et valeurs

Type	Nombre de bits	Interval de valeurs
boolean	1	deux valeurs true et false
byte	8	un entier entre -128 et 127
short	16	un entier entre $-2^{15} = -32768$ et $2^{15} - 1 = 32767$
int	32	un entier entre $-2^{31} \approx -2.1 \cdot 10^9$ et $2^{31} - 1 \approx 2.1 \cdot 10^9$
long	64	un entier entre $-2^{63} \approx -9.2 \cdot 10^{18}$ et $2^{63} - 1 \approx 9.2 \cdot 10^{18}$
char	16	caractère unicode, il y a 65536 codes
float	32	nombre flottant norme IEEE
double	64	nombre flottant norme IEEE

Variables

○ Affectation

Syntaxe : lvalue = expression

lvalue est une expression qui doit délivrer une variable (par exemple un identificateur de variable, élément de tableau...., mais pas une constante)

Exemples

```
1 | short population ;  
2 | population = 30000;
```

```
1 | short population = 1.000.000;
```

```
1 | long nbParticules = 10.000.000.000;
```

```
1 | long nbParticules = 10.000.000.000L;
```

```
1 | char lettre = 'c' ;
```

```
1 | boolean test = true ;  
2 | test = false ;
```

Variables

○ Conversion (Transtypage ou cast)

On a la situation suivante :

```
1 <type1> <nom1> = <valeur1>;  
2 <type2> <nom2> = <nom1>;
```

La *conversion* ou *cast* peut rester *implicite* si le <type₁> est « moins fort » que le <type₂>

```
1 int i = 10;  
2 double x = i;
```

La conversion doit devenir *explicite* si le <type₁> est « strictement plus fort » que le <type₂> : il faut indiquer au compilateur d'effectuer la conversion.

```
1 double x = 3.1416;  
2 int i = (int)x;
```

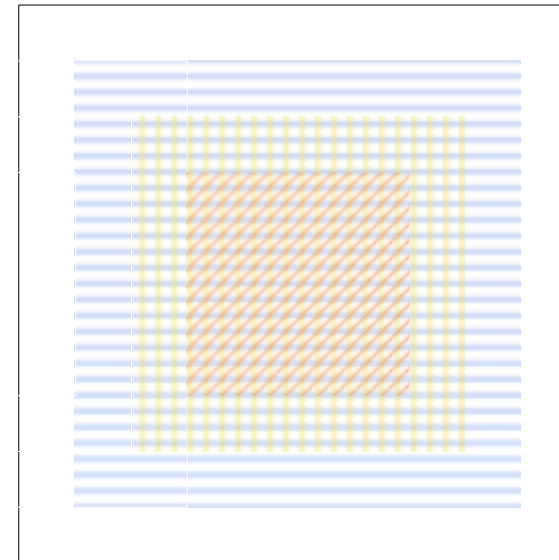
Variables

○ Visibilité d'une variable dans un bloc

Un bloc permet de grouper un ensemble d'instructions en lui donnant la forme syntaxique d'une seule instruction

Les variables déclarées dans un bloc interne **ne** sont **pas** connues dans un bloc plus externe.

```
1  int a,b=10;  
2  {  
3      int d=2*b;  
4      {  
5          int e=b+d;  
6          a=e*d;  
7          {  
5              int g= b+ d*e;  
6          }  
6      }  
7  }
```



Expressions et opérateurs

- Type d'une expression

Le code suivant est-il correct ?

```
1  int i = 5, j;  
2  double x = 5.0;  
3  j=i/2;  
4  j=x/2;
```

```
1  double x=2.75;  
2  int y = (int) x * 2;  
3  int z = (int) (x *2);
```

Quelles sont les valeurs de *y* et *z* ?

Expressions et opérateurs

- Opérateurs unaires

Opérateur	action	exemples
+	signe positif	+a; +7
-	signe négatif	-a; -(a-b); -7
!	négation logique	!(a<b);
++	affectation et incrément de 1	n++;++n;
--	affectation et incrément de 1	n++; --i;

Expressions et opérateurs

○ Opérateurs binaires

Opérateur	action	exemples
*	multiplication	$a * i$
/	division	$n/10$
%	reste de la division entière	$k \% n$
+	addition	$1+2$
-	soustraction	$x-5$
<	strictement inférieur	$i < n$
<=	inférieur ou égal	$i \leq n$
>	strictement supérieur	$i > n$
>=	supérieur ou égal	$i \geq n$
==	égalité	$i == j$
!=	différent	$i != j$
&	conjonction (et logique)	$(i < j) \& (i < n)$
	disjonction (ou logique)	$(i < j) (i < n)$
&&	conjonction optimisée	$(i < j) \&\& (i < n)$
	disjonction optimisée	$(i < j) (i < n)$
=	affectation	$x = 10; x = n;$
+=, -=	affectation et incrément	$i += 2; j -= 4$

Expressions et opérateurs

○ Exemple

```
1  int i=2, j = i++;  
2  i=2;  
3  j= ++i;
```

Attention à ne pas utiliser = pour faire un test d'égalité!

Expressions et opérateurs

- Opérateur conditionnel ternaire

```
1 | result = uneCondition ? value1 : value2;
```

Si le test (une expression booléenne) `uneCondition` est vérifié, **alors** la variable `result` prend la valeur `value1`,
sinon elle prend la valeur `value2`.

```
1 | double x, y, r=1.0;  
2 | ...  
3 | boolean interieur = x*x + y*y < r ? true : false
```

Structures conditionnelles

- La structure **if ... then ... else**

```
1  if ( <expression booléenne> )
2      <bloc d'instructions à exécuter
3          si la condition est satisfaite>
4  else
5      <bloc d'instructions à exécuter
6          si la condition n'est pas satisfaite>
```

```
1  int gains,payment,encaissement,invest;
2  // opérations qui modifient la variable gains
3  ...
4  if (gains<0)
5      payment = gains;
6  else if (gains > 10) {
7      encaissement = 10;
8      invest = gains-10;
9  }
10 else
11     encaissement = gains;
```

Structures conditionnelles

○ Choix multiples

```
1  int choix;  
2  ...  
3  // l'utilisateur modifie la valeur de choix  
4  ...  
5  switch (choix) {  
6      case 1:  
7          //instructions pour le choix 1  
8          ...  
9          break;  
10     case 2:  
11         //instructions pour le choix 2  
12         ...  
13         break;  
14     default  
15         // instructions dans le reste des cas  
16         ...  
17 }
```

Le switch peut s'effectuer sur deux types de variables : int et char. Depuis la version 7, on peut aussi utiliser une chaîne de caractères

Structures répétitives : boucles

○ Boucle **for**

```
1  for (<initialisation> ; <condition de fin> ;  
2      <mise à jour des valeurs> )  
3      <bloc d'instructions>
```

que se passe-t-il ?

```
1  for ( ; ; ) {  
2      // instructions  
3  }
```

Structures répétitives : boucles

○ Boucle **for**

un exemple classique :

```
1 | int n=10;  
2 | for (int i=0; i< n; i++ ) {  
3 |     // instructions  
4 | }
```

```
1 | int n=10;  
2 | for (int i=0, j=n; j< i; i++, j--){  
3 |     // instructions  
4 | }
```

Structures répétitives : boucles

- Boucle **while**

```
1 | while (<condition>)  
2 |   <bloc d'instructions>
```

Le bloc d'instructions est exécuté **tant que** la condition est satisfaite.

Structures répétitives : boucles

- Boucle **do** ... **while**

```
1  do  
2      <bloc d'instructions>  
3  while(<condition>) ;
```

Attention : ne **pas** oublier le **;** à la fin du `while` !

Structures répétitives : boucles

- Choix du type de la boucle

- généralement, si on connaît le nombre d'itérations, on utilise une boucle **for**.
- qu'est-ce qui est plus adapté ?
- qu'est-ce qui sera le plus facile à lire pour un autre lecteur ?

ex :

- chercher un élément dans un tableau ?
- chercher l'élément le plus grand d'un tableau ?
- déterminer si un nombre est premier ?

Séance 2

Les méthodes en Java

Du code réutilisable à la mise en pratique

Qu'est-ce qu'une méthode ?

Définition

Une **méthode** est un bloc d'instructions regroupées sous un même nom et pouvant être exécutées lorsqu'on l'appelle.

Pourquoi utiliser une méthode ?

- Éviter de répéter le même code
- Organiser le programme
- Rendre le code plus lisible
- Réutiliser un traitement plusieurs fois

Qu'est-ce qu'une méthode ?

Exemple

Sans méthode

```
System.out.println("Bonjour !");  
System.out.println("Bonjour !");  
System.out.println("Bonjour !");
```

Avec une méthode

```
static void bonjour() {  
    System.out.println("Bonjour !");  
}
```

Déclaration d'une méthode

Syntaxe générale

```
</> Java

static typeRetour nomMethode(parametres) {
    // instructions
}
```

Exemple

```
</> Java

static void afficherBonjour() {
    System.out.println("Bonjour !");
}
```

Élément	Signification
static	Permet d'appeler la méthode depuis main sans créer d'objet
void	La méthode ne retourne aucune valeur
afficherBonjour	Nom de la méthode
()	Liste des paramètres
{ }	Corps de la méthode

Appel d'une méthode

Une méthode n'est pas exécutée simplement parce qu'elle est déclarée. Elle doit être **appelée**.

</> Java

```
static void afficherBonjour() {  
    System.out.println("Bonjour !");  
}  
  
public static void main(String[] args) {  
    afficherBonjour();  
}
```

Avec plusieurs appels

</> Java

```
public static void main(String[] args) {  
  
    afficherBonjour();  
    afficherBonjour();  
    afficherBonjour();  
  
}
```

À retenir

Déclarer une méthode ≠ exécuter une méthode.

Paramètres

Pourquoi utiliser des paramètres ?

Les paramètres permettent de transmettre des informations à une méthode.

```
</> Java

static void afficherNom(String nom) {
    System.out.println("Bonjour " + nom);
}
```

Appel :

```
</> Java

afficherNom("Alice");
```

Résultat :

```
Bonjour Alice
```

Avec plusieurs paramètres

```
</> Java

static void afficherPersonne(String nom, int age) {
    System.out.println(nom + " a " + age + " ans.");
}
```

Appel :

```
</> Java

afficherPersonne("Alice", 20);
```

Résultat :

```
Alice a 20 ans.
```

À retenir : Un **paramètre** permet à une méthode de recevoir une valeur lors de son appel.

Paramètres : paramètre et argument

Déclaration :

```
</> Java

static void afficherAge(int age) {
    System.out.println(age);
}
```

Appel :

```
</> Java

afficherAge(25);
```

- age est un paramètre.
- 25 est un argument.

Schéma

Déclaration		Appel
int age	← paramètre	25 ← argument
↓		↓
afficherAge(int age)		afficherAge(25)

Valeur de retour

Jusqu'ici, nos méthodes faisaient une action mais **ne renvoyaient aucune valeur** :

```
</> Java

static void afficherBonjour() {
    System.out.println("Bonjour");
}
```

Une méthode peut également **retourner une valeur**.

Exemple

```
</> Java

static int addition(int a, int b) {
    return a + b;
}
```

```
</> Java

int resultat = addition(5, 3);

System.out.println(resultat);
```

À retenir : return permet de **renvoyer une valeur** à l'endroit où la méthode a été appelée.

void ou valeur de retour ?

Méthode void

Elle effectue une action mais **ne retourne pas de valeur**.

```
</> Java

static void afficherBonjour() {
    System.out.println("Bonjour");
}
```

Appel :

```
</> Java

afficherBonjour();
```

Méthode avec retour

Elle calcule ou produit une valeur.

```
</> Java

static int carre(int x) {
    return x * x;
}
```

Appel :

```
</> Java

int resultat = carre(5);
```

void ou valeur de retour ?

Comparaison

void	Avec valeur de retour
Effectue une action	Produit une valeur
Pas de return avec valeur	Utilise return
<code>afficherBonjour()</code>	<code>carre(5)</code>
<code>void</code>	<code>int, double, String, etc.</code>

Portée des variables

Définition

La **portée d'une variable** correspond à la partie du programme dans laquelle cette variable peut être utilisée.

```
</> Java

static void calcul() {

    int nombre = 10;

    System.out.println(nombre);
}
```

nombre peut être utilisé à l'intérieur de la méthode.

```
</> Java

static void calcul() {

    int nombre = 10;
}

public static void main(String[] args) {

    System.out.println(nombre); // Erreur
}
```

✗ nombre n'est pas accessible dans main.

À retenir : Une variable locale est accessible uniquement dans le **bloc où elle a été déclarée** et ses blocs internes.

Exemple complet

Exemple qui réunit **méthode + paramètres + retour + appel**.

```
</> Java

public class Calcul {

    static int addition(int a, int b) {
        int resultat = a + b;
        return resultat;
    }

    public static void main(String[] args) {

        int somme = addition(10, 5);

        System.out.println(somme);
    }
}
```

Exercices d'application